

Dokumentasi Software Architecture Project - QuickBite

Kelompok 5 :

- Edwin Winarto (2802445431)
- Joseline Fransiska Wijaya (2802455722)
- Kelly Aurelya Tiona (2802455325)
- Michelle Santoso (2802452582)
- Vincent Juvento (2802446011)

Lampiran :

- Github Link : <https://github.com/oopsitscel/QuickBite>
- Project Presentation : <https://canva.link/eni8sfk1uh6a6f5>
- Video Teaser / Video Demo Aplikasi
- Rencana Postingan sosial media sebagai showcase project anda apabila diposting pada LinkedIn dan Instagram
- Project Report
- Exhibition Registration Evidence

1. Project Overview

QuickBite adalah platform pemesanan makanan berbasis web yang dirancang untuk menghubungkan pelanggan, chef, dan admin dalam satu sistem terintegrasi. Website ini menyediakan fitur untuk melihat menu makanan, melakukan pemesanan, mengelola pesanan, serta memantau aktivitas operasional secara lebih efisien.

Sistem dibangun menggunakan teknologi frontend berbasis HTML, CSS, dan JavaScript, serta backend menggunakan framework NestJS dengan TypeScript untuk mendukung performa aplikasi yang scalable dan terstruktur.

QuickBite memiliki beberapa role pengguna, yaitu:

- Guest → dapat melihat menu dan informasi makanan.
- Customer → dapat memesan makanan dan melihat status pesanan.
- Chef → mengelola serta memproses pesanan makanan.
- Admin → mengatur data menu, kategori, dan operasional sistem.

Dengan adanya pembagian role tersebut, QuickBite mampu mensimulasikan alur bisnis food ordering system secara lebih realistis dan terorganisir.

2. Latar Belakang

Dalam era digital saat ini, industri restoran memerlukan sistem informasi yang efisien untuk mengelola operasional bisnis secara tepat dan terstruktur. Banyak restoran tradisional masih menghadapi berbagai permasalahan, seperti kesulitan dalam mengelola menu, kategori produk, pesanan customer, serta koordinasi antar staff. Selain itu, proses pemesanan yang masih dilakukan secara manual sering menyebabkan kesalahan pencatatan, duplikasi

pesanan, keterlambatan pelayanan, hingga kehilangan data pesanan. Kondisi tersebut dapat menurunkan kualitas pelayanan dan mengurangi kepuasan pelanggan.

Seiring berkembangnya teknologi dan meningkatnya kebutuhan masyarakat terhadap layanan digital yang praktis, restoran dituntut untuk memiliki sistem yang mampu mengotomatisasi proses operasional dan meningkatkan efisiensi kerja. Oleh karena itu, QuickBite dirancang sebagai solusi terintegrasi berbasis web yang menggabungkan frontend yang user-friendly dengan backend yang robust dan scalable. Aplikasi ini memungkinkan customer untuk melihat menu dan melakukan pemesanan makanan secara online dengan mudah dan cepat. Di sisi lain, admin dapat mengelola data menu dan kategori produk, chef dapat melihat pesanan yang ditugaskan serta memperbarui status pesanan, dan sistem dapat membantu mengatur workflow pesanan dari awal hingga selesai secara lebih terorganisir.

Selain bertujuan untuk membantu digitalisasi proses bisnis restoran, proyek ini juga menjadi implementasi pembelajaran pengembangan aplikasi web full-stack dengan memanfaatkan teknologi frontend dan backend modern, penerapan role management, serta integrasi sistem client-server untuk menciptakan aplikasi yang efisien, responsif, dan mudah digunakan.

3. Tujuan QuickBite

- a. Membangun sistem manajemen restoran yang terintegrasi antara frontend dan backend untuk mempermudah proses pemesanan dan pengelolaan operasional restoran.
- b. Mempermudah customer dalam melakukan pemesanan makanan secara online dengan proses yang cepat, praktis, dan terstruktur.
- c. Membantu admin dan chef dalam mengelola menu, kategori, serta workflow pesanan secara real-time agar operasional restoran menjadi lebih efisien.
- d. Mengurangi kesalahan pencatatan, duplikasi pesanan, dan kehilangan data yang sering terjadi pada sistem pemesanan manual.
- e. Mengimplementasikan konsep software architecture dan design pattern dalam pengembangan aplikasi agar sistem lebih terstruktur dan mudah dipelihara.
- f. Menciptakan aplikasi yang scalable, maintainable, dan mudah dikembangkan di masa depan dengan menerapkan best practices seperti authentication, authorization, validasi data, dan error handling.

4. How to Run the Application

4.1. Import Database

Sebelum aplikasi dijalankan, database perlu diimpor ke dalam MySQL agar seluruh tabel dan data awal tersedia. Pengguna terlebih dahulu membuat database baru dengan nama *quickbite_db* menggunakan MySQL Workbench atau phpMyAdmin. Setelah database berhasil dibuat, file *quickbite_db.sql* diimpor ke dalam database tersebut melalui menu

Import. Setelah proses import selesai, seluruh struktur tabel beserta data awal akan otomatis tersimpan di dalam database sehingga siap digunakan oleh aplikasi.

4.2. Konfigurasi Backend

Setelah database berhasil diimpor, buka folder backend pada project QuickBite kemudian install seluruh dependency menggunakan perintah berikut.

```
npm install
```

Selanjutnya, buat file .env pada folder backend dan isi dengan konfigurasi berikut.

```
PORT=5000
```

```
DATABASE_URL="mysql://root:@localhost:3306/quickbite_db"
```

```
JWT_SECRET=quickbite_secret
```

Konfigurasi tersebut digunakan untuk menentukan port backend, koneksi ke database MySQL, serta secret key yang digunakan dalam proses autentikasi menggunakan JSON Web Token (JWT).

4.3. Menjalankan Backend

Setelah konfigurasi selesai dilakukan, generate Prisma Client menggunakan perintah berikut :

```
npx prisma generate
```

Kemudian jalankan backend menggunakan perintah:

```
npm run start:dev
```

Apabila proses berhasil, backend akan berjalan pada alamat <http://localhost:5000> dan siap menerima request dari frontend.

4.4. Menjalankan Frontend

Frontend QuickBite dikembangkan menggunakan HTML, CSS, dan JavaScript sehingga tidak memerlukan proses build. Pengguna cukup membuka folder frontend menggunakan Visual Studio Code, kemudian menjalankan file guest.html menggunakan ekstensi Live Server. Setelah Live Server dijalankan, aplikasi dapat diakses melalui browser dan siap digunakan.

5. Software Architecture

5.1. Client-Server Architecture

QuickBite menggunakan arsitektur Client-Server Architecture yang memisahkan sistem menjadi tiga komponen utama:

i. Frontend (Client Layer)

Frontend QuickBite dibangun menggunakan HTML untuk struktur, CSS untuk styling, dan JavaScript untuk interaktivitas. Frontend bertugas untuk:

- Menampilkan user interface yang responsif dan user-friendly
- Mengambil input dari user melalui form dan interaksi
- Mengirimkan HTTP request ke backend menggunakan fetch API
- Mengelola state aplikasi di sisi client seperti shopping cart dan user session

Contoh implementasi di QuickBite:

```
const response = await
fetch("http://localhost:5000/menu");
const menus = await response.json();
```

Frontend QuickBite terdiri dari beberapa halaman:

- login.html - untuk autentikasi user
- register.html - untuk pendaftaran user baru
- guest.html - untuk guest user yang ingin melihat menu tanpa login
- customer.html - untuk customer yang sudah login dengan fitur pesan
- chef.html - untuk chef untuk melihat dan update pesanan
- admin.html - untuk admin untuk manage menu dan user

ii. Backend (Server Layer)

Backend QuickBite dibangun menggunakan NestJS, sebuah framework TypeScript-based yang powerful untuk membangun server-side application. Backend bertanggung jawab untuk:

- Menjalankan semua business logic aplikasi
- Mengimplementasikan authentication dan authorization
- Melakukan validasi data yang ketat sebelum menyimpan ke database
- Mengakses database dan mengembalikan data yang diminta

Teknologi yang digunakan:

- NestJS

Digunakan sebagai framework backend untuk membangun server-side application yang terstruktur, scalable, dan maintainable.

- TypeScript
Digunakan sebagai bahasa pemrograman utama untuk memberikan type-safety serta mempermudah proses development dan debugging.
- JWT (JSON Web Token)
Digunakan untuk proses authentication dan authorization agar sistem keamanan pengguna lebih terjamin.
- Bcrypt
Digunakan untuk melakukan hashing password sehingga data password pengguna tersimpan dengan lebih aman.
- Prisma ORM
Digunakan sebagai Object Relational Mapping (ORM) untuk mempermudah pengelolaan database dan operasi data pada aplikasi.

iii. Database Layer

Database QuickBite menggunakan MySQL sebagai RDBMS (Relational Database Management System).

Database menyimpan entitas-entitas utama:

- User
Menyimpan informasi pengguna sistem beserta role yang dimiliki, seperti ADMIN, CHEF, dan CUSTOMER.
- Category
Menyimpan data kategori menu untuk membantu pengelompokan makanan atau minuman.
- Menu
Menyimpan detail item menu, seperti nama menu, harga, stok, deskripsi, dan estimasi waktu memasak.
- Order
Menyimpan informasi pesanan customer, termasuk status pesanan, customer yang melakukan order, dan chef yang ditugaskan.
- OrderDetail
Menyimpan detail item-item yang terdapat pada setiap pesanan, termasuk jumlah item dan subtotal harga.

5.2. Layered Architecture

Backend QuickBite dipisahkan menjadi tiga layer yang jelas:

i. Controller Layer

Controller layer adalah entry point untuk semua HTTP request dari frontend. Setiap controller handle routing untuk endpoint spesifik dan melakukan request/response handling.

Contoh dari MenuController:

```
@Controller('menu')
export class MenuController {
  @Get()
  findAll() {
    return this.menuService.findAll();
  }

  @Post()
  @Roles('ADMIN', 'CHEF')
  create(@Body() dto: CreateMenuDto) {
    return this.menuService.create(dto);
  }
}
```

Tanggung jawab controller:

- Routing HTTP request ke method yang tepat
- Validasi parameter dan body request
- Return response yang sesuai

ii. Service Layer

Service layer berisi semua business logic aplikasi. Service handle proses data, validasi logic, dan orchestration antara berbagai operasi.

Contoh dari MenuService:

```
@Injectable()
export class MenuService {
  constructor(private prisma: PrismaService) {}

  async create(dto: CreateMenuDto) {
    // Validate menu name tidak duplikat
    const existingMenu = await
    this.prisma.menu.findFirst({
```

```

        where: { name: dto.name }
    });
    if (existingMenu) throw new BadRequestException('Menu
name
        already exists');

    // Validate category ada
    const category = await
this.prisma.category.findUnique({
        where: { id: dto.categoryId }
    });
    if (!category) throw new BadRequestException('Category
not found');

    // Create menu
    return this.prisma.menu.create({ data: dto });
}
}

```

Tanggung jawab service:

- Menjalankan business logic
- Melakukan validasi data dan business rules
- Mengkoordinasikan multiple database operations
- Handling error dan exception

Pemisahan logic ke dalam service layer sangat penting karena dapat menjaga controller tetap clean dan hanya berfokus pada proses routing dan handling request. Selain itu, business logic yang dipisahkan ke service layer menjadi lebih mudah untuk diuji secara independen, dapat digunakan kembali oleh beberapa controller, serta mempermudah proses pengembangan dan modifikasi logic tanpa harus mengubah struktur controller yang sudah ada.

iii. Data Access Layer

Data Access Layer menggunakan Prisma ORM untuk berkomunikasi dengan database. Prisma menyediakan type-safe query builder yang membuat database operation menjadi lebih aman dan maintainable.

Contoh penggunaan Prisma:

```

// Create
await this.prisma.order.create({
  data: {
    customerId: dto.customerId,
    totalPrice: totalPrice,
    details: { create: orderDetails }
  }
});

// Read
const menus = await this.prisma.menu.findMany({
  include: { category: true }
});

// Update
await this.prisma.menu.update({
  where: { id: menuId },
  data: { stock: { decrement: quantity } }
});

// Delete
await this.prisma.menu.delete({
  where: { id: menuId }
});

```

Keuntungan menggunakan Prisma:

- Query type-safe: error akan terdeteksi pada compile time
- Mengurangi SQL injection vulnerability
- Query lebih readable dan maintainable
- Auto-generate dari schema

6. Design Pattern Implementation

a. MVC Pattern (Modified)

QuickBite menerapkan konsep MVC (Model-View-Controller) yang dimodifikasi untuk context NestJS:

- Model: Prisma Schema dan MySQL Database merepresentasikan struktur data
- View: HTML/CSS/JavaScript frontend merepresentasikan tampilan

- Controller: NestJS Controller menghandle HTTP request dan koordinasi
- Pemisahan ini memastikan bahwa presentasi terpisah dari logika bisnis dan storage.

b. Repository Pattern

Prisma secara implicit mengimplementasikan Repository Pattern yaitu pattern yang membuat abstraksi antara business logic dan data access. Service tidak perlu tahu bagaimana SQL query dijalankan, cukup panggil method Prisma dan biarkan Prisma handle detail teknisnya.

Keuntungan:

- Database abstraction: Mudah untuk mengganti database di masa depan
- Maintenance yang mudah: Perubahan query hanya di satu tempat
- Reduce coupling: Service tidak tightly bound ke database implementation

c. Dependency Injection Pattern

NestJS secara built-in mengimplementasikan Dependency Injection. Service tidak membuat instance sendiri, tetapi menerima dependency melalui constructor:

```
@Injectable()
export class MenuService {
  constructor(private prisma: PrismaService) {}
}
```

Keuntungan:

- Loose coupling antara service dan dependency
- Mudah untuk testing (dependency dapat di-mock)
- Service reusable dengan berbagai implementasi dependency

d. Module Pattern

QuickBite menggunakan Module Pattern untuk memisahkan fitur berdasarkan concern:

- PrismaModule: Database connection dan Prisma service
- AuthModule: Authentication dan JWT handling
- UsersModule: User management
- CategoriesModule: Category management
- MenusModule: Menu management
- OrdersModule: Order management

Setiap module berisi controller, service, dan dependencies yang dibutuhkan. Keuntungan:

- Scalability: Fitur baru dapat ditambahkan sebagai module baru
- Code organization: Code yang relevan terletak di satu tempat
- Independent testing: Setiap module dapat di-test secara terpisah

e. Singleton Pattern

Singleton Pattern adalah design pattern yang memastikan bahwa suatu class hanya memiliki satu instance selama aplikasi berjalan, dan memberikan global point of access ke instance tersebut. Di QuickBite, NestJS secara default mengimplementasikan Singleton Pattern untuk semua service yang dideklarasikan dengan decorator `@Injectable()`.

Contoh implementasi:

```
@Injectable()
export class PrismaService extends PrismaClient implements
OnModuleInit {
  async onModuleInit() {
    await this.$connect();
  }
}
```

PrismaService hanya di-instantiate satu kali di seluruh aplikasi dan di-inject ke semua service yang membutuhkan:

```
@Injectable()
export class MenuService {
  constructor(private prisma: PrismaService) {} // Singleton
instance
}

@Injectable()
export class OrdersService {
  constructor(private prisma: PrismaService) {} // Sama dengan
instance di atas
}
```

Keuntungan Singleton Pattern di QuickBite adalah memory efficiency karena hanya ada satu instance database connection, konsistensi state karena semua service berbagi instance yang sama, dan controlled access ke resource yang berharga seperti database connection.

f. State Pattern

State Pattern adalah design pattern yang memungkinkan object mengubah behavior ketika internal state-nya berubah. Di QuickBite, State Pattern diimplementasikan melalui

Order Status yang memiliki beberapa state berbeda dan transisi antar state yang diatur dengan ketat.

Order memiliki lima state yang mungkin:

```
enum OrderStatus {  
    PENDING        // State awal order  
    COOKING         // Order sedang dimasak  
    READY           // Order siap diambil/dikirim  
    COMPLETED      // Order selesai  
    CANCELLED       // Order dibatalkan  
}
```

Setiap state memiliki transisi yang valid ke state lain:

```
const validTransitions: Record<OrderStatus, OrderStatus[]>  
= {  
    // Dari PENDING bisa ke COOKING atau CANCELLED  
    PENDING : [OrderStatus.COOKING, OrderStatus.CANCELLED],  
    // Dari COOKING hanya bisa ke READY  
    COOKING : [OrderStatus.READY],  
    // Dari READY hanya bisa ke COMPLETED  
    READY : [OrderStatus.COMPLETED],  
    COMPLETED : [], // COMPLETED adalah final state  
    CANCELLED : [],  // CANCELLED adalah final state  
};
```

Implementasi State Pattern ini memastikan bahwa order status tidak dapat berubah ke state yang invalid:

```
const allowed = validTransitions[order.status];  
if (!allowed.includes(status)) {  
    throw new BadRequestException(`Invalid status transition  
from ${order.status} to ${status}`);  
}
```

Keuntungan State Pattern di QuickBite adalah business rule enforcement karena transisi state diatur ketat, workflow predictability karena state machine yang jelas dan terstruktur, dan easy state management karena logic state change terpusat di satu tempat.

7. Authentication & Authorization

QuickBite mengimplementasikan security yang robust menggunakan:

a. JWT Authentication

- User melakukan login dengan email dan password
- Backend validate password menggunakan bcrypt
- Jika valid, backend generate JWT token yang berisi user id, email, dan role
- Frontend menyimpan JWT token di localStorage
- Setiap API request membawa JWT token di header Authorization
- Backend verify JWT token dan validate user

b. Role-Based Access Control

QuickBite memiliki tiga role utama dengan hak akses dan tanggung jawab yang berbeda, yaitu:

- ADMIN
Memiliki akses penuh untuk mengelola data pengguna (users), kategori menu, data menu, serta melakukan assign chef pada setiap pesanan yang masuk.
- CHEF
Bertanggung jawab untuk membuat dan memperbarui menu, serta mengubah status pesanan sesuai proses pengerjaan makanan.
- CUSTOMER
Dapat melihat menu, membuat pesanan makanan, dan memantau riwayat maupun status pesanan yang dimiliki.

Implementasi menggunakan decorator dan guard:

```
@Patch('/:id/role')
@UseGuards(JwtAuthGuard, RolesGuard)
@ApiBearerAuth()
@Roles('ADMIN')
updateRole(
  @Param('id') id: string,
  @Body() dto: UpdateRoleDto,
) {
}
```

8. API Endpoints

QuickBite memiliki 17 endpoint yang terbagi menjadi beberapa kategori:

- a. Authentication (2 endpoints)
 - POST /auth/register - Register user baru
 - POST /auth/login - Login dan dapatkan JWT token
- b. User Management (3 endpoints)
 - GET /users - Ambil semua user (ADMIN only)
 - GET /users/:id - Ambil detail user (ADMIN only)
 - PATCH /users/:id/promote-chef - Ubah role user (ADMIN only)
- c. Category Management (2 endpoints)
 - GET /categories - Ambil semua kategori (Public)
 - POST /categories - Buat kategori baru (ADMIN only)
- d. Menu Management (5 endpoints)
 - GET /menu - Ambil semua menu (Public)
 - GET /menu/:id - Ambil detail menu (Public)
 - POST /menu - Buat menu baru (ADMIN, CHEF)
 - PATCH /menu/:id - Update menu (ADMIN, CHEF)
 - PATCH /menu/:id/availability - Toggle ketersediaan menu (ADMIN, CHEF)
- e. Order Management (5 endpoints)
 - POST /orders - Buat order baru (CUSTOMER)
 - GET /orders - Ambil order sesuai role
 - GET /orders/:id - Ambil detail order
 - POST /orders/:id/assign-chef - Assign chef ke order (ADMIN)
 - PATCH /orders/:id/status - Update status order (CHEF)

Endpoint	Access	Deskripsi
POST /auth/register	Public	Sistem backend menerima data registrasi berupa nama, email, dan password dari user baru. Backend melakukan validasi untuk memastikan email belum terdaftar sebelumnya dan password memenuhi kriteria kompleksitas (harus mengandung huruf besar, huruf kecil, angka, dan karakter spesial). Setelah validasi berhasil, password di-hash menggunakan bcrypt untuk keamanan, lalu user baru disimpan ke database dengan role default

		CUSTOMER. Sistem mengembalikan JWT token untuk memungkinkan user login otomatis dan data user yang baru terdaftar.
POST /auth/login	Public	Sistem backend menerima email dan password dari user yang ingin login. Backend mencari user di database berdasarkan email, kemudian memverifikasi kecocokan password menggunakan bcrypt compare. Jika kredensial valid, sistem generate JWT token yang berisi informasi user dan role untuk keperluan autentikasi di endpoint-endpoint berikutnya. Token dan data user kemudian dikirim kembali ke frontend.
GET /users	Admin	Endpoint ini dilindungi dengan JWT authentication dan role-based access control sehingga hanya admin yang dapat mengakses. Sistem backend mengambil daftar lengkap semua user yang terdaftar di database dan menampilkannya kepada admin untuk tujuan monitoring dan manajemen user.
GET /users/:id	Admin	Endpoint ini dilindungi dengan autentikasi JWT dan kontrol akses berbasis role sehingga hanya admin yang diizinkan mengakses. Sistem backend mengambil data detail user spesifik berdasarkan ID yang diminta dan mengembalikan informasi lengkap user tersebut atau mengirim error jika user tidak ditemukan.
PATCH /users/:id/role	Admin	Endpoint ini dilindungi dengan JWT authentication dan role-based access control yang membatasi hanya admin yang dapat melakukan operasi ini. Sistem backend menerima informasi role baru dari request, memverifikasi user dengan ID tersebut ada di database. Data user yang sudah diupdate kemudian dikembalikan.
GET /categories	Public	Endpoint publik yang dapat diakses oleh siapa saja tanpa memerlukan autentikasi. Sistem backend mengambil daftar lengkap semua kategori menu dari database dan menampilkannya ke user agar mereka dapat melihat kategori-kategori yang tersedia.
POST /categories	Admin	Endpoint ini dilindungi dengan JWT authentication dan role-based access control sehingga hanya admin yang diizinkan menambah kategori baru. Sistem backend menerima data kategori (nama dan

		deskripsi), memvalidasi inputan, lalu menyimpan kategori baru ke database. Sistem mengembalikan data kategori yang baru dibuat.
GET /menu	Public	Endpoint publik yang dapat diakses oleh siapa saja tanpa memerlukan login. Sistem backend mengambil daftar lengkap semua item menu dari database beserta informasi relasi seperti kategori, harga, stok, dan deskripsi menu untuk ditampilkan ke user.
GET /menu/:id	Public	Endpoint publik yang dapat diakses tanpa autentikasi. Sistem backend mengambil data detail satu item menu berdasarkan ID spesifik yang diminta dan menampilkan informasi lengkap menu tersebut atau error jika menu tidak ditemukan.
POST /menu	Admin, Chef	Endpoint ini dilindungi dengan JWT authentication dan role-based access control sehingga hanya admin atau chef yang dapat membuat item menu baru. Sistem backend menerima data menu (nama, harga, stok, deskripsi, estimasi waktu masak, URL gambar, kategori), melakukan validasi untuk memastikan kategori ada dan nama menu tidak duplikat, lalu menyimpan menu baru ke database. Sistem mengembalikan data menu yang telah dibuat.
PATCH /menu/:id	Admin, Chef	Endpoint ini dilindungi dengan JWT authentication dan role-based access control yang membatasi hanya admin atau chef yang dapat melakukan perubahan. Sistem backend menerima data update menu, memverifikasi menu dengan ID tersebut ada, memvalidasi jika ada perubahan nama untuk memastikan tidak terjadi duplikasi, kemudian mengupdate properti menu di database (nama, harga, stok, deskripsi, dsb). Data menu yang sudah diupdate dikembalikan.
PATCH /menu/:id/availability	Admin, Chef	Endpoint ini dilindungi dengan JWT authentication dan role-based access control yang membatasi akses hanya untuk admin atau chef. Sistem backend menerima request untuk mengubah status ketersediaan menu, melakukan toggle pada status isAvailable menu di database (mengaktifkan atau menonaktifkan menu), lalu mengembalikan data menu dengan status ketersediaan yang sudah diperbarui.

POST /orders	Customer	Endpoint ini dilindungi dengan JWT authentication dan role-based access control sehingga hanya user dengan role CUSTOMER yang dapat membuat order. Sistem backend menerima data order berisi ID customer dan daftar item menu dengan kuantitas, melakukan validasi komprehensif untuk setiap menu (memastikan tersedia, stok mencukupi, harga valid), menghitung total harga keseluruhan order, menyimpan order dengan status PENDING dan detail setiap item ke database, serta secara otomatis mengurangi stok menu sesuai jumlah pesanan. Data order lengkap beserta detail item dikembalikan.
GET /orders	Authenticated	Endpoint ini dilindungi dengan JWT authentication untuk memastikan user sudah login. Sistem backend mengambil daftar order dari database dengan logika filtering berbasis role: admin dapat melihat semua order di sistem, customer hanya melihat order milik diri sendiri, sedangkan chef hanya melihat order yang telah ditugaskan kepada mereka. Data order dikembalikan terurut dari yang paling baru.
GET /orders/:id	Public	Endpoint saat ini dapat diakses tanpa memerlukan autentikasi (tidak ada JWT guard di controller). Sistem backend mengambil data detail order spesifik berdasarkan ID yang diminta beserta relasi terkait seperti informasi customer, chef yang ditugaskan, dan detail item menu dalam order tersebut. Data order lengkap dikembalikan atau error jika order tidak ditemukan.
PATCH /orders/:id/ assign-chef	Admin	Endpoint ini dilindungi dengan JWT authentication dan role-based access control sehingga hanya admin yang dapat menugaskan chef ke order. Sistem backend menerima ID chef dari request, melakukan validasi untuk memastikan order ada dengan status PENDING dan belum ada chef yang ditugaskan sebelumnya, memverifikasi bahwa user dengan ID chef tersebut benar-benar memiliki role CHEF, lalu mengupdate order dengan menambahkan chef assignment. Data order yang sudah di-assign dikembalikan.
PATCH /orders/:id/ status	Chef	Endpoint ini dilindungi dengan JWT authentication dan role-based access control yang membatasi akses hanya untuk user dengan role CHEF. Sistem backend

		menerima status baru dari request, melakukan validasi untuk memastikan order ada dan sudah ada chef yang ditugaskan, kemudian memverifikasi bahwa transisi status valid sesuai aturan workflow (PENDING dapat berubah ke COOKING atau CANCELLED, COOKING dapat berubah ke READY, READY dapat berubah ke COMPLETED). Jika valid, status order di-update di database dan data order dikembalikan dengan status terbaru.
--	--	---

9. Hasil dan Implementasi

9.1. Frontend Implementation

Frontend QuickBite mengimplementasikan API call pattern yang konsisten.

```
async function fetchAPI(endpoint, options = {}) {
  const config = {
    method: options.method || "GET",
    headers: {
      "Content-Type": "application/json",
      ...(options.headers || {}),
    },
  };

  if(options.body) {
    config.body = options.body;
  }

  const response = await
fetch(`http://localhost:5000${endpoint}`, config);
  const data = await response.json();

  if (!response.ok) {
    throw new Error(data.message);
  }

  return data;
}
```

Fitur-fitur frontend:

- Menu browsing dan filtering berdasarkan kategori
- Shopping cart dengan increment/decrement quantity
- Order management dengan status tracking
- Role-based UI (different pages untuk ADMIN, CHEF, CUSTOMER)

9.2. Backend Implementation

Backend mengimplementasikan validasi yang comprehensive.

Order Creation Validation:

```
async create(dto: CreateOrderDto) {
  // 1. Validate customer
  const customer = await this.prisma.user.findUnique({
    where: { id: dto.customerId }
  });
  if (!customer) throw new NotFoundException('Customer not found');
  if (customer.role !== Role.CUSTOMER) throw new
BadRequestException('Only customer can create orders');

  // 2. Validate setiap menu item
  for (const item of dto.items) {
    const menu = await this.prisma.menu.findUnique({
      where: { id: item.menuId }
    });
    if (!menu) throw new NotFoundException('Menu not found');
    if (!menu.isAvailable) throw new BadRequestException('Menu
unavailable');
    if (menu.stock < item.quantity) throw new
BadRequestException('Insufficient stock');
  }

  // 3. Create order dan update stock
  // ...
}
```

9.3. Database Schema

Prisma schema mendefinisikan entitas dan relasi.

```
model User {
```

```

id String @id @default(uuid())
email String @unique
password String
role Role // ADMIN, CHEF, CUSTOMER

customerOrders Order[] @relation("CustomerOrders")
chefOrders Order[] @relation("ChefOrders")
}

model Menu {
  id String @id @default(uuid())
  name String
  price Float
  stock Int
  categoryId String
  category Category @relation(fields: [categoryId], references: [id])
  orderItems OrderDetail[]
}

model Order {
  id String @id @default(uuid())
  status OrderStatus // PENDING, COOKING, READY, COMPLETED
  customerId String
  chefId String?
  details OrderDetail[]
}

```

10. Kesimpulan

QuickBite merupakan aplikasi manajemen restoran berbasis web yang berhasil mengimplementasikan berbagai konsep software architecture dan design pattern dalam proses pengembangannya. Sistem ini dirancang untuk membantu operasional restoran menjadi lebih modern, terstruktur, dan efisien melalui integrasi antara frontend dan backend. Dengan adanya sistem ini, proses pengelolaan menu, kategori makanan, pemesanan

customer, hingga workflow pesanan dapat dilakukan secara lebih terorganisir dibandingkan dengan sistem manual.

Dalam implementasinya, QuickBite menggunakan client-server architecture yang memisahkan frontend dan backend secara jelas dengan komunikasi melalui REST API. Pendekatan ini membuat proses pengembangan menjadi lebih fleksibel karena frontend dan backend dapat dikembangkan secara terpisah tanpa saling bergantung secara langsung. Selain itu, backend aplikasi juga menerapkan layered architecture dengan pembagian Controller, Service, dan Data Access Layer agar struktur kode lebih rapi, mudah dipahami, dan lebih mudah dipelihara. Selain itu, QuickBite juga menerapkan berbagai design pattern seperti MVC, Repository Pattern, Dependency Injection, dan Module Pattern juga berhasil diterapkan pada sistem untuk meningkatkan kualitas arsitektur aplikasi. Dengan penerapan design pattern tersebut, kode program menjadi lebih modular, reusable, dan scalable sehingga mempermudah proses pengembangan maupun penambahan fitur di masa depan.

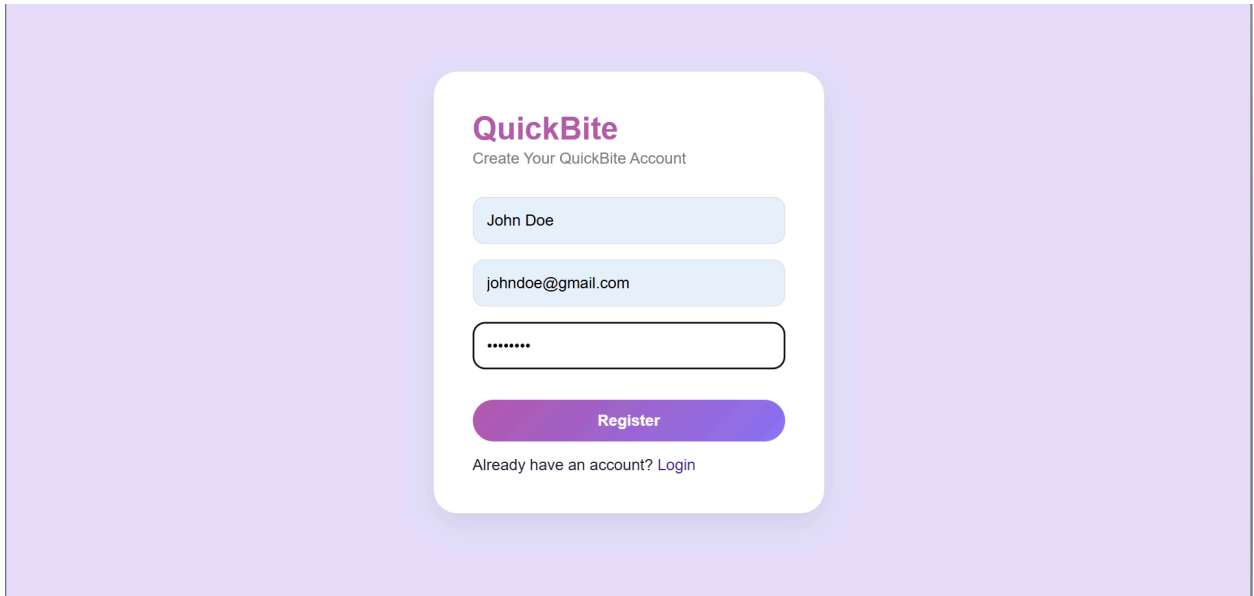
Dari sisi keamanan, QuickBite telah menerapkan JWT authentication dan role-based authorization untuk memastikan keamanan akses pengguna. Setiap pengguna hanya dapat mengakses fitur sesuai role yang dimiliki, yaitu ADMIN, CHEF, dan CUSTOMER. Selain itu, sistem juga menerapkan validasi data pada setiap layer untuk menjaga integritas data dan mengurangi kemungkinan terjadinya error atau data tidak valid pada aplikasi.

Secara keseluruhan, QuickBite berhasil menjadi solusi digital untuk manajemen restoran yang tidak hanya membantu proses operasional menjadi lebih efektif dan efisien, tetapi juga menjadi implementasi nyata penerapan konsep software engineering modern dalam pengembangan aplikasi web full-stack. Dengan fondasi arsitektur yang sudah dibangun, aplikasi ini masih memiliki potensi besar untuk dikembangkan lebih lanjut melalui penambahan fitur seperti payment integration, delivery tracking, review system, notification system, dan analytics dashboard di masa depan.

11. Testing

a. Register

Halaman register digunakan untuk membuat akun baru dengan mengisi data dan credential yang diperlukan, seperti nama, email, dan password. Setelah akun berhasil dibuat, pengguna dapat login ke dalam sistem dan akan diarahkan ke halaman sesuai role masing-masing.



The image shows a registration form for 'QuickBite' on a light purple background. The form is a white rounded rectangle with the title 'QuickBite' in purple and the subtitle 'Create Your QuickBite Account' in grey. It contains three input fields: a name field with 'John Doe', an email field with 'johndoe@gmail.com', and a password field with masked characters. Below the fields is a purple 'Register' button and a link 'Already have an account? Login'.

QuickBite
Create Your QuickBite Account

John Doe

johndoe@gmail.com

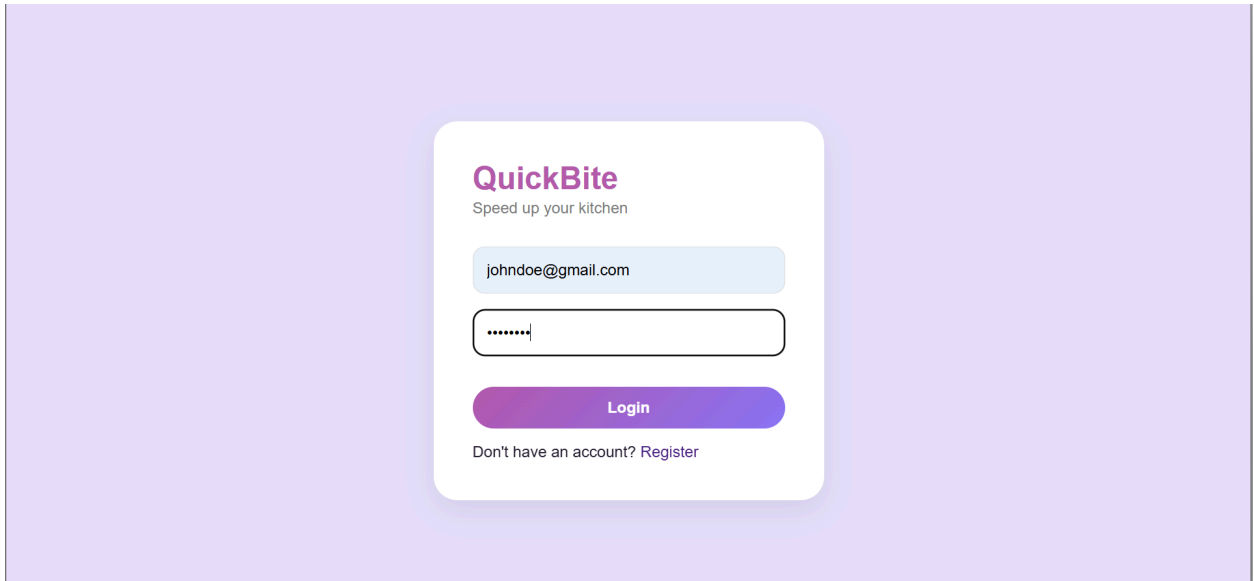
.....

Register

Already have an account? [Login](#)

b. Login

Halaman login digunakan untuk mengakses akun pengguna dengan memasukkan credential yang valid, seperti email dan password. Setelah berhasil login, pengguna akan diarahkan ke halaman yang berbeda sesuai dengan role yang dimiliki, seperti ADMIN, CHEF, atau CUSTOMER.



The image shows a login form for 'QuickBite' on a light purple background. The form is a white rounded rectangle with the title 'QuickBite' in purple and the subtitle 'Speed up your kitchen' in grey. It contains two input fields: an email field with 'johndoe@gmail.com' and a password field with masked characters. Below the fields is a purple 'Login' button and a link 'Don't have an account? Register'.

QuickBite
Speed up your kitchen

johndoe@gmail.com

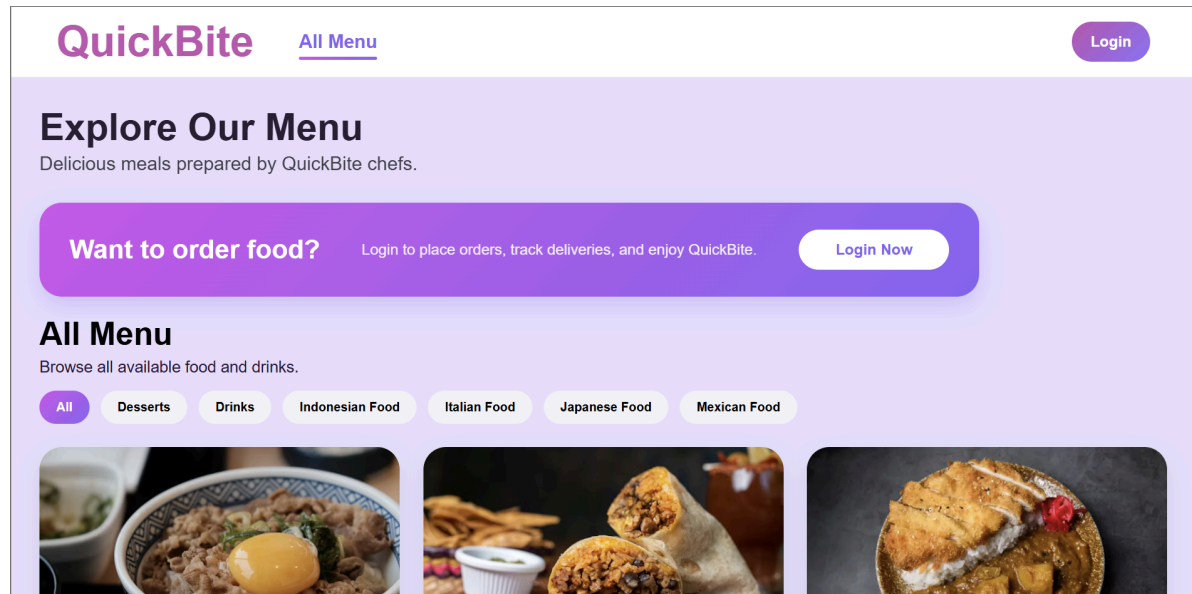
.....

Login

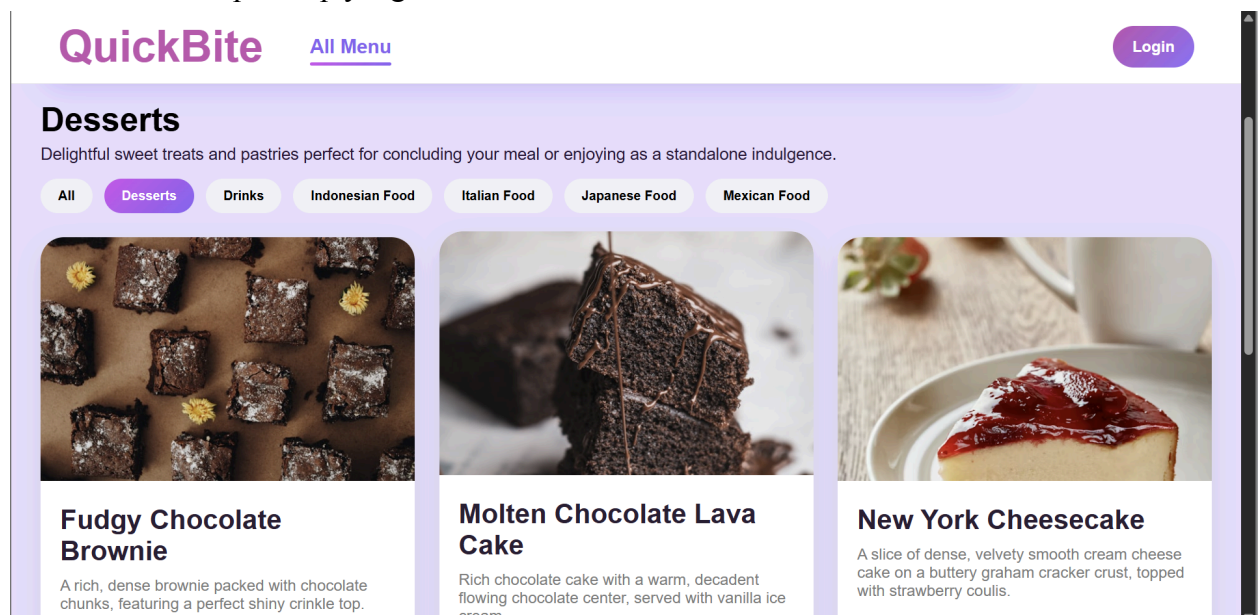
Don't have an account? [Register](#)

c. Guest Page

Sebelum melakukan login atau register, pengguna akan terlebih dahulu masuk ke halaman guest page, di mana pengguna dapat melihat informasi dan daftar menu yang tersedia secara umum tanpa perlu memiliki akun.

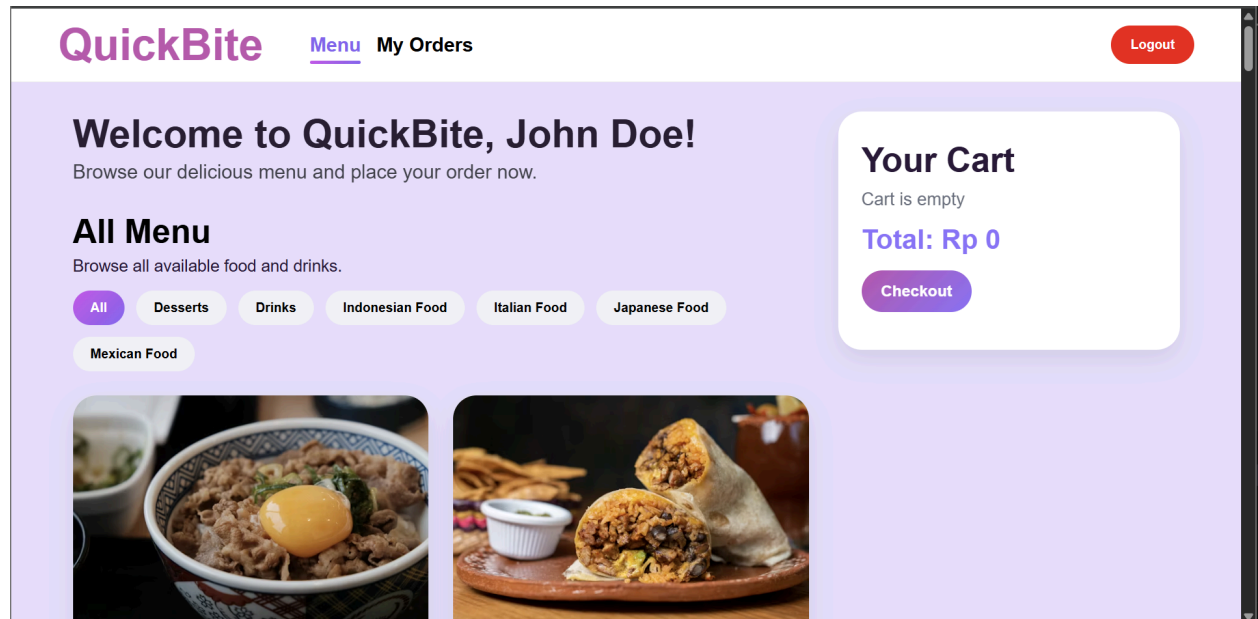


Ketika kategori **Desserts** dipilih, sistem akan menampilkan daftar menu yang hanya termasuk ke dalam kategori dessert, sehingga pengguna dapat lebih mudah mencari dan melihat makanan penutup yang tersedia.

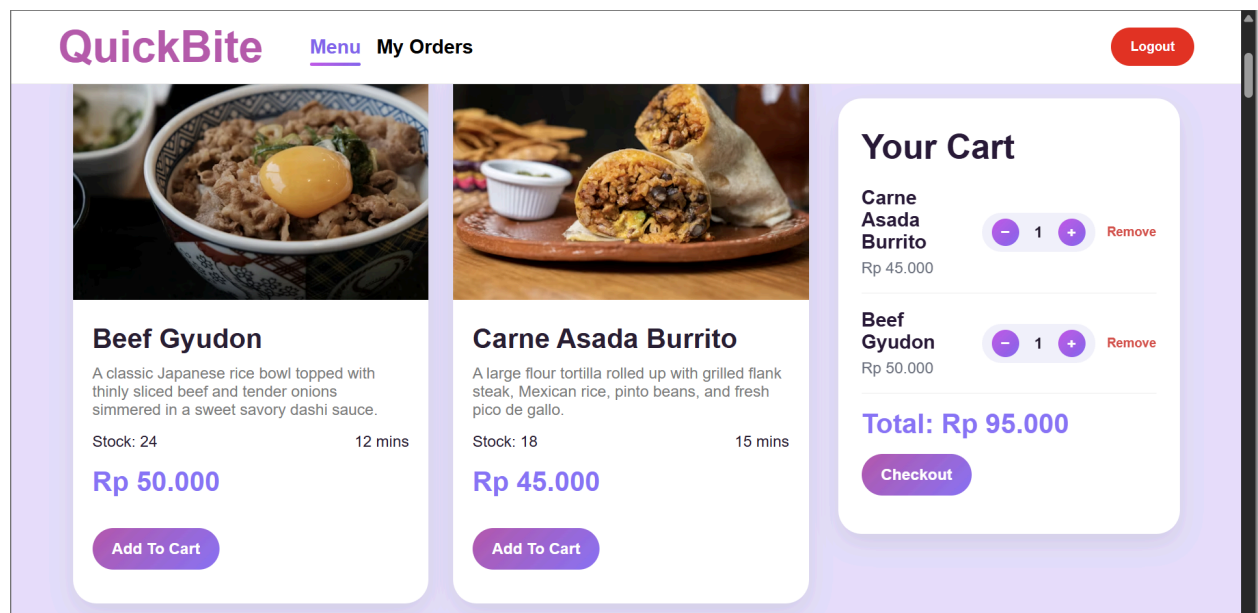


d. Customer Page

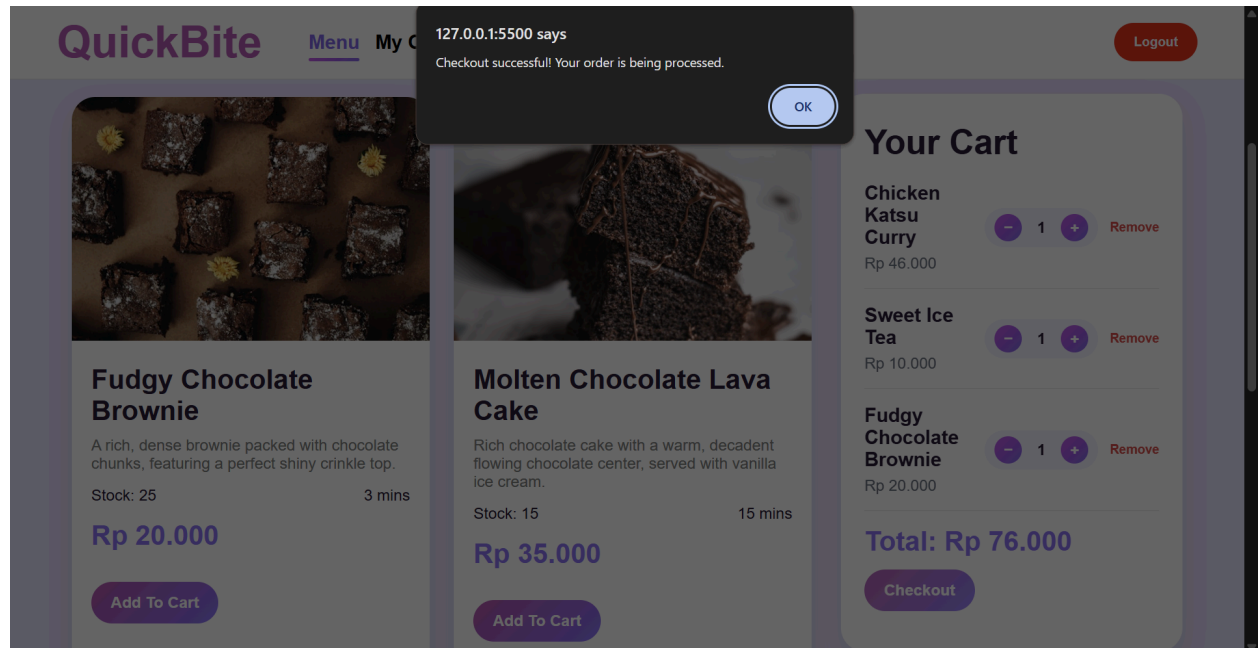
Setelah berhasil login sebagai **CUSTOMER**, pengguna akan diarahkan ke halaman **Customer Page** yang berisi fitur pemesanan makanan, melihat menu, serta memantau status dan riwayat pesanan.



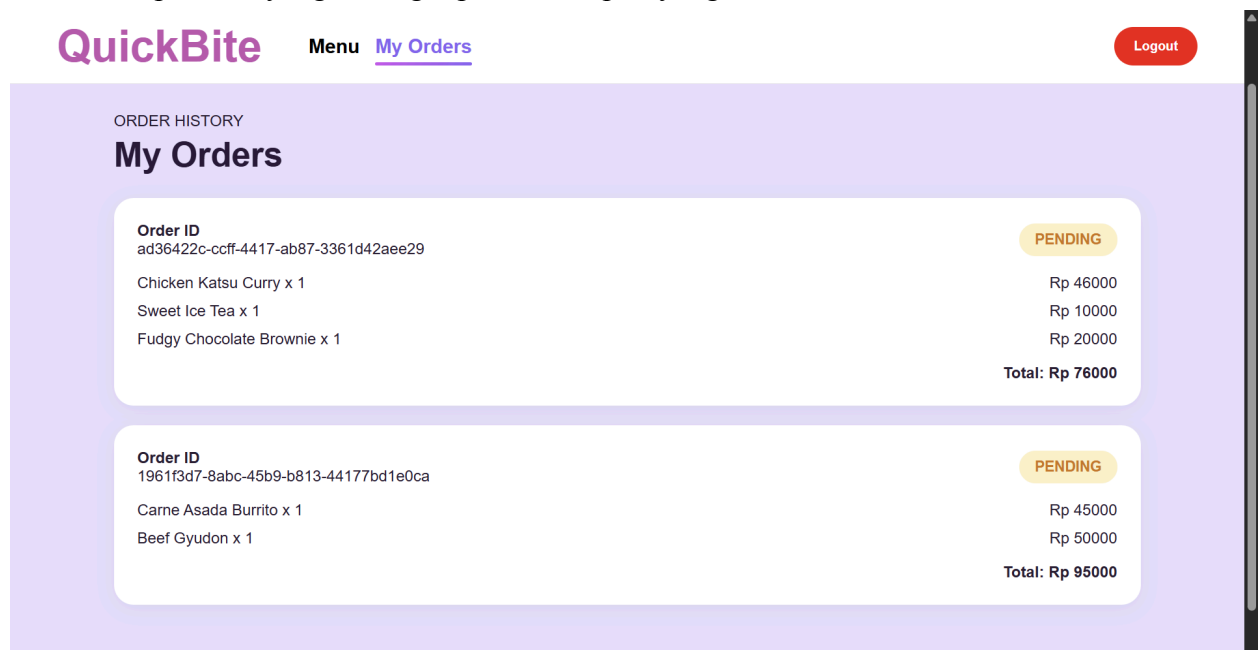
Ketika tombol **Add to Cart** ditekan, item yang dipilih akan langsung muncul pada bagian **Your Cart** di halaman yang sama. Sistem akan otomatis menghitung subtotal setiap item dan total keseluruhan pesanan secara real-time. Pengguna juga dapat menambah atau mengurangi jumlah item menggunakan tombol (+) dan (-), serta menghapus item dari cart menggunakan tombol **Remove**.



Ketika tombol **Checkout** ditekan, seluruh item yang terdapat pada **Your Cart** akan otomatis diproses dan masuk ke dalam daftar order milik customer. Sistem akan menyimpan detail pesanan, total pembayaran, serta status order untuk diproses lebih lanjut oleh chef dan admin. Setelah pesanan berhasil dibuat, order tersebut akan langsung tampil pada halaman **My Orders** sehingga customer dapat melihat dan memantau status pesannya.

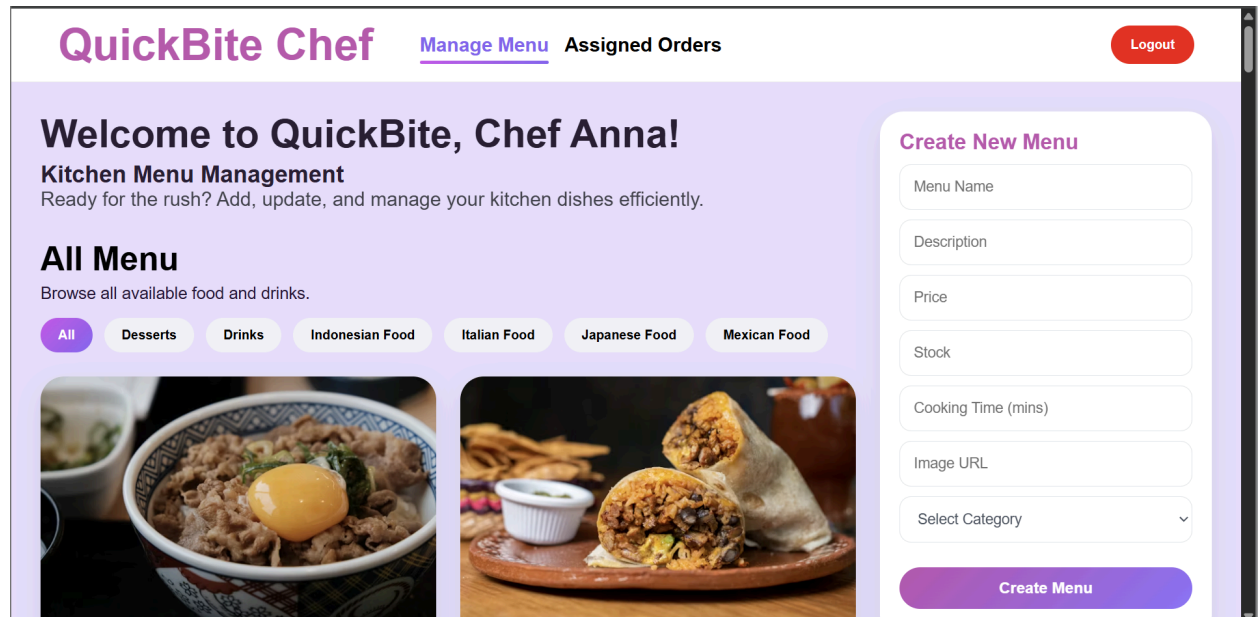


Halaman **My Orders** digunakan untuk menampilkan seluruh pesanan yang telah dibuat oleh customer. Pada halaman ini, pengguna dapat melihat detail setiap order, total pembayaran, serta status pesanan yang sedang diproses maupun yang sudah selesai.

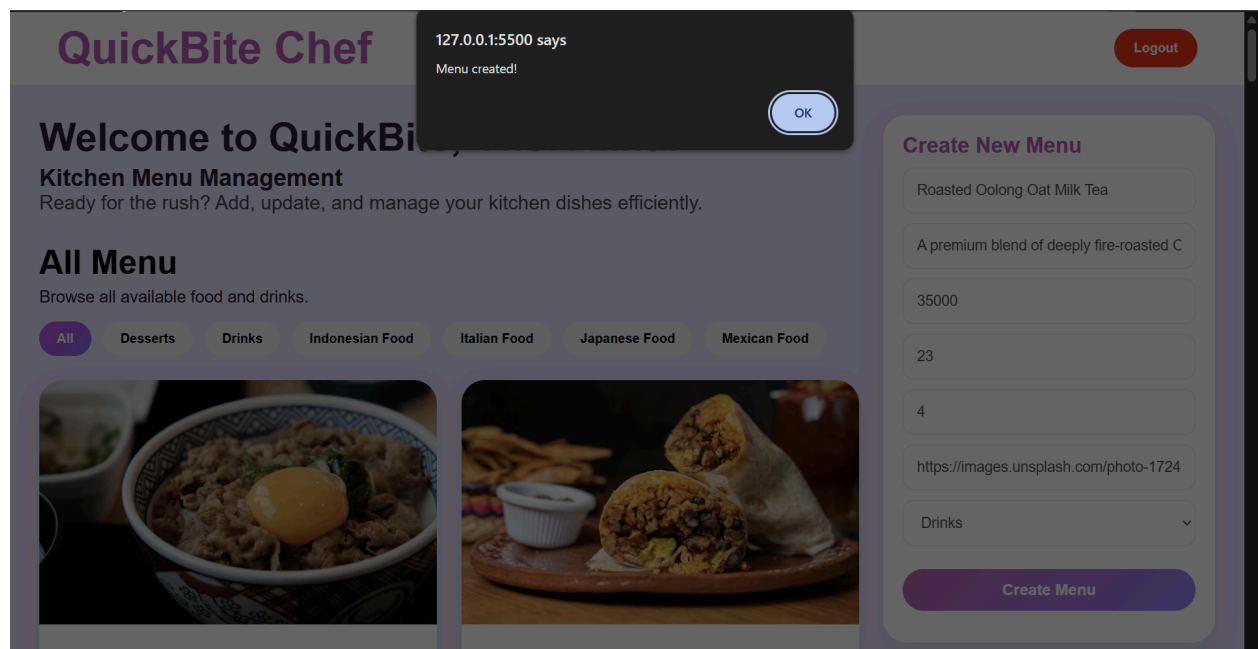


e. Chef Page

Setelah berhasil login sebagai **CHEF**, pengguna akan diarahkan ke halaman **Chef Page** yang digunakan untuk melihat daftar pesanan yang masuk, memperbarui status pesanan, serta mengelola proses pembuatan makanan.

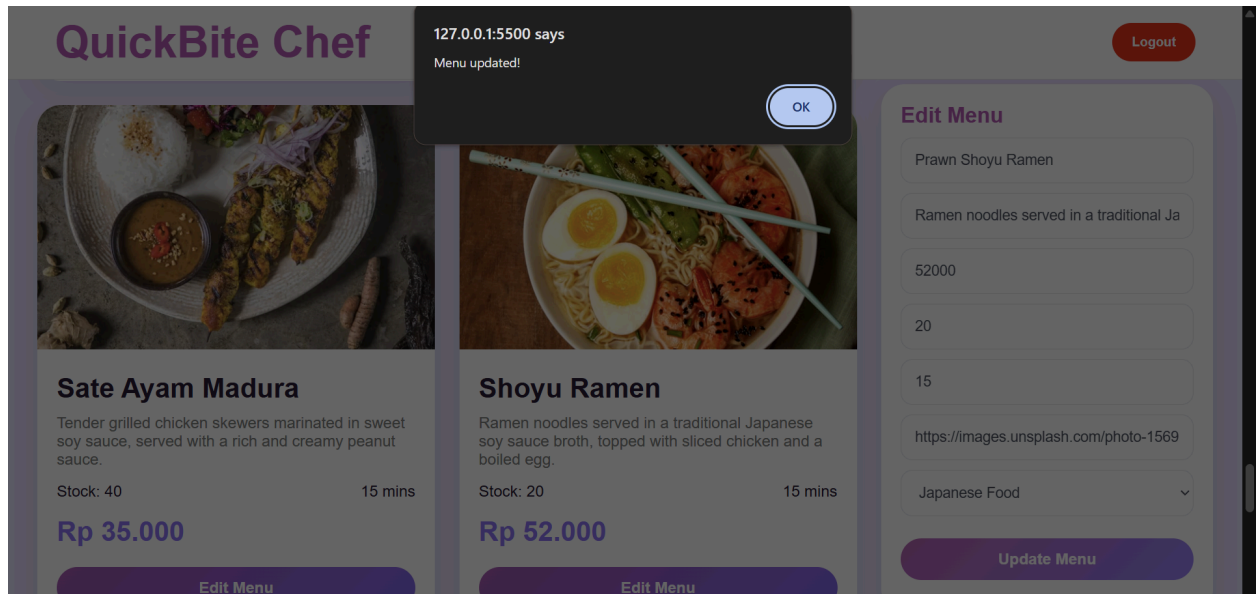


Pada halaman **Chef Page**, chef dapat melakukan fitur **Create New Menu** untuk menambahkan menu baru ke dalam sistem. Chef perlu mengisi beberapa informasi menu, seperti nama menu, deskripsi makanan atau minuman, harga, stok, estimasi waktu memasak, gambar menu, serta kategori makanan. Setelah data berhasil diisi dan tombol **Create Menu** ditekan, menu baru akan otomatis ditambahkan ke daftar menu.

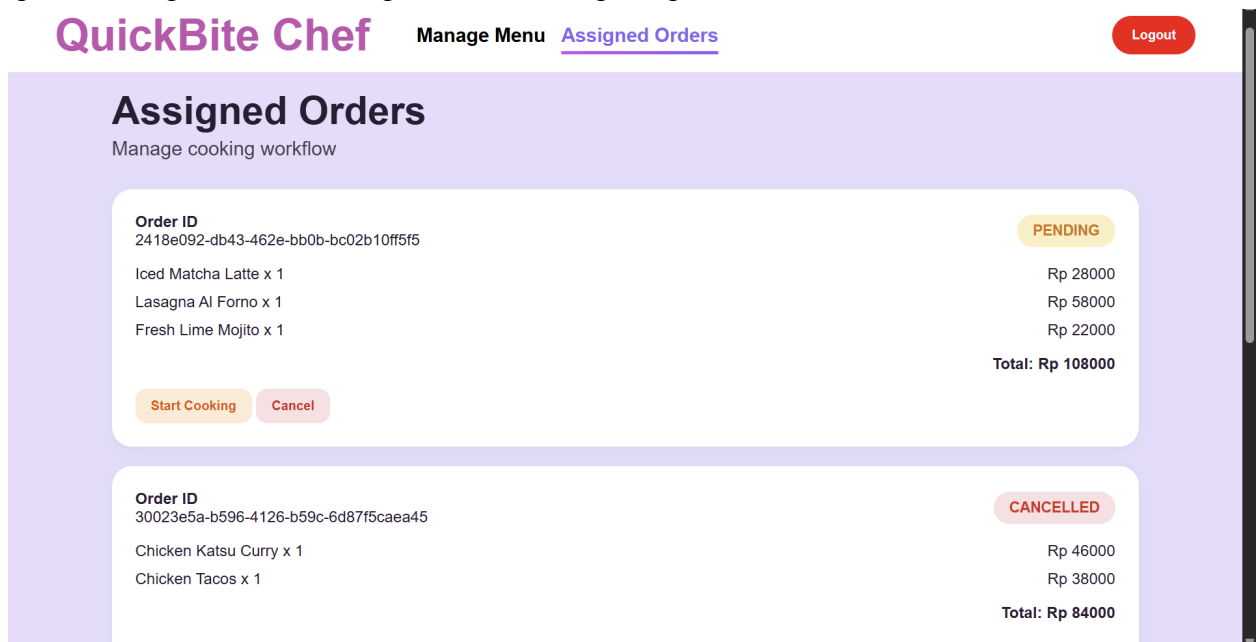


Chef dapat memperbarui informasi menu yang sudah ada. Ketika tombol **Edit Menu** ditekan, data menu akan otomatis tampil pada form di sebelah kanan sehingga chef dapat langsung mengubah detail seperti nama menu, deskripsi, harga, stok, estimasi waktu memasak,

gambar, dan kategori menu. Setelah perubahan selesai dilakukan, chef dapat menekan tombol **Update Menu** untuk menyimpan perubahan ke dalam sistem.



Pada halaman **Assigned Orders**, chef dapat melihat seluruh pesanan yang telah di-assign kepadanya, termasuk pesanan dengan status **Pending**, **Cooking**, **Completed**, maupun **Cancelled**. Chef dapat memulai proses pembuatan makanan dengan menekan tombol **Start Cooking**. Selain itu, chef juga dapat membatalkan pesanan menggunakan fitur **Cancel Order** apabila terdapat kendala atau pesanan tidak dapat diproses.



Ketika tombol **Start Cooking** ditekan, status pesanan akan berubah dari **Pending** menjadi **Cooking**. Setelah itu, tombol aksi akan berubah menjadi **Mark Ready** yang digunakan oleh chef untuk menandakan bahwa makanan telah selesai dibuat dan siap disajikan atau dikirim.

Assigned Orders

Manage cooking workflow

Order ID

2418e092-db43-462e-bb0b-bc02b10ff5f5

Iced Matcha Latte x 1

Lasagna Al Forno x 1

Fresh Lime Mojito x 1

COOKING

Rp 28000

Rp 58000

Rp 22000

Total: Rp 108000

Mark Ready

Order ID

30023e5a-b596-4126-b59c-6d87f5caea45

Chicken Katsu Curry x 1

Chicken Tacos x 1

CANCELLED

Rp 46000

Rp 38000

Total: Rp 84000

Ketika tombol **Mark Ready** ditekan, status pesanan akan berubah menjadi **Ready** sebagai tanda bahwa makanan telah selesai dibuat dan siap disajikan atau dikirim. Setelah itu, tombol aksi akan berubah menjadi **Complete** untuk menandakan bahwa pesanan telah benar-benar selesai diproses.

Assigned Orders

Manage cooking workflow

Order ID

2418e092-db43-462e-bb0b-bc02b10ff5f5

Iced Matcha Latte x 1

Lasagna Al Forno x 1

Fresh Lime Mojito x 1

READY

Rp 28000

Rp 58000

Rp 22000

Total: Rp 108000

Complete

Order ID

30023e5a-b596-4126-b59c-6d87f5caea45

Chicken Katsu Curry x 1

Chicken Tacos x 1

CANCELLED

Rp 46000

Rp 38000

Total: Rp 84000

Ketika tombol **Complete** ditekan, status pesanan akan berubah menjadi **Completed** sebagai tanda bahwa seluruh proses pesanan telah selesai dilakukan.

QuickBite Chef

Manage MenuAssigned Orders

Logout

Assigned Orders

Manage cooking workflow

Order ID
2418e092-db43-462e-bb0b-bc02b10ff5f5

COMPLETED

Iced Matcha Latte x 1
Lasagna Al Forno x 1
Fresh Lime Mojito x 1

Rp 28000
Rp 58000
Rp 22000
Total: Rp 108000

Order ID
30023e5a-b596-4126-b59c-6d87f5caea45

CANCELLED

Chicken Katsu Curry x 1
Chicken Tacos x 1

Rp 46000
Rp 38000
Total: Rp 84000

f. Admin Page

Setelah berhasil login sebagai **ADMIN**, pengguna akan diarahkan ke halaman **Admin Page** yang digunakan untuk mengelola data pengguna, kategori, menu, serta mengatur dan memantau workflow pesanan dalam sistem.

QuickBite Admin

OrdersUsersCategoriesMenu

Logout

Welcome to QuickBite, Admin Budi Wijaya!

Orders Management

Assign chefs and monitor orders

Order ID
ad36422c-ccff-4417-ab87-3361d42aee29

No Chef Assigned

PENDING

Chicken Katsu Curry x 1
Sweet Ice Tea x 1
Fudgy Chocolate Brownie x 1

Rp 46000
Rp 10000
Rp 20000
Total: Rp 76000

Select ChefAssign Chef

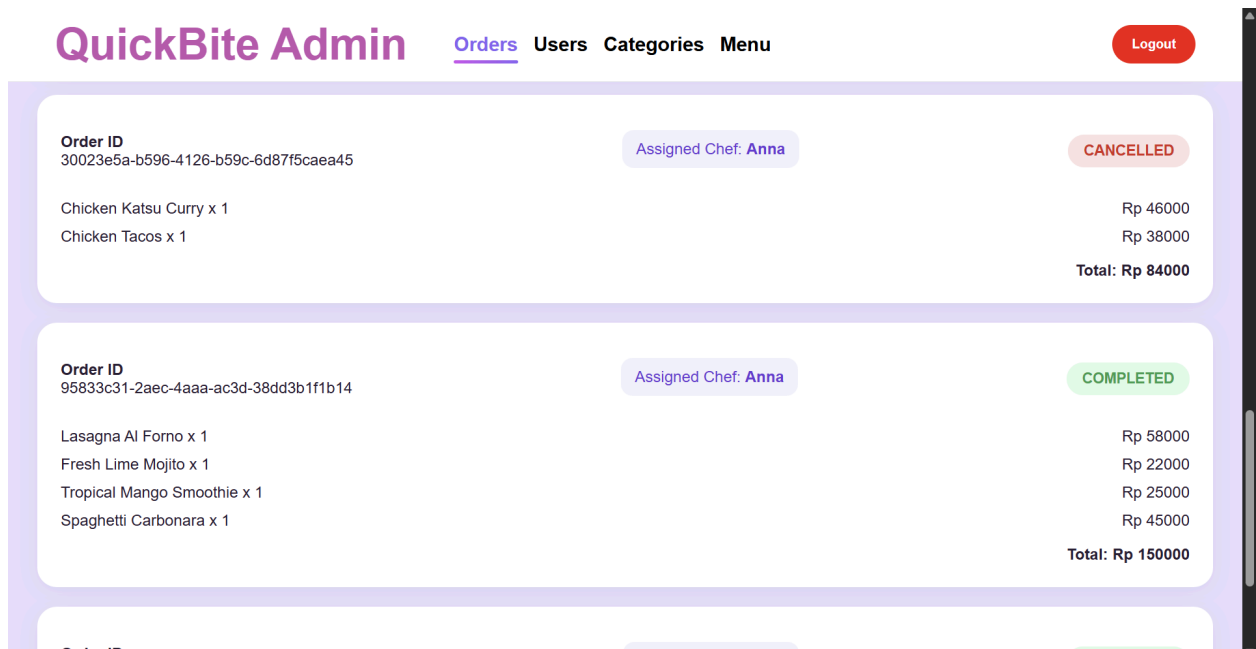
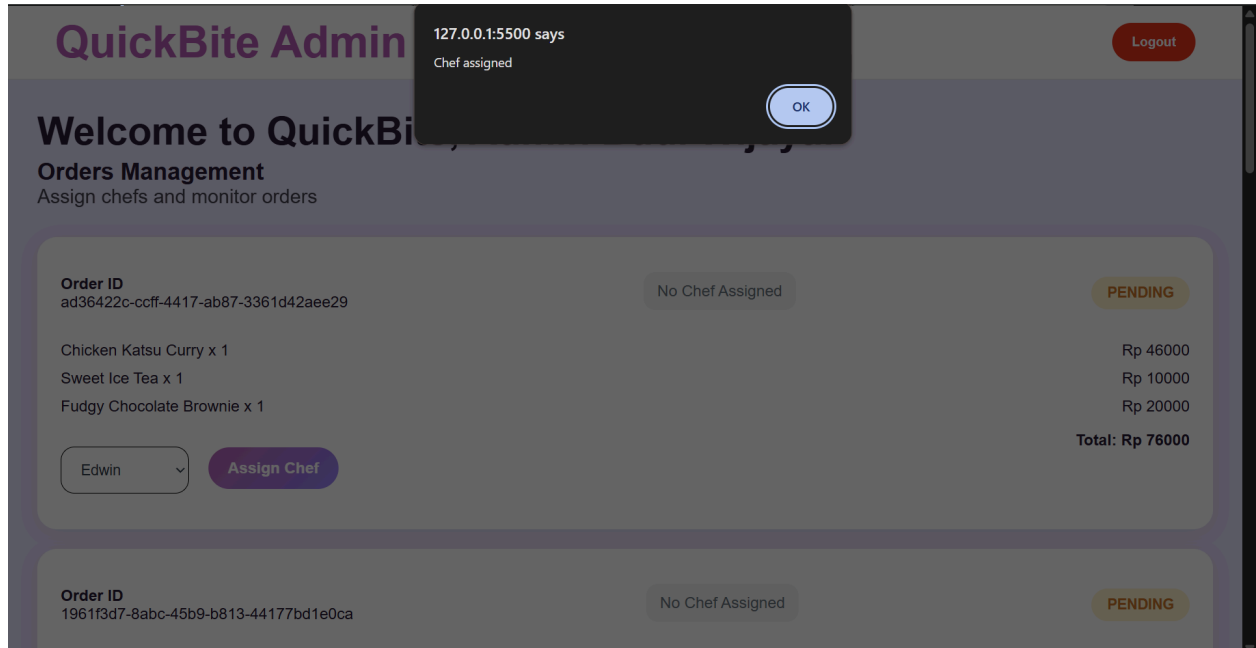
Order ID
1961f3d7-8abc-45b9-b813-44177bd1e0ca

No Chef Assigned

PENDING

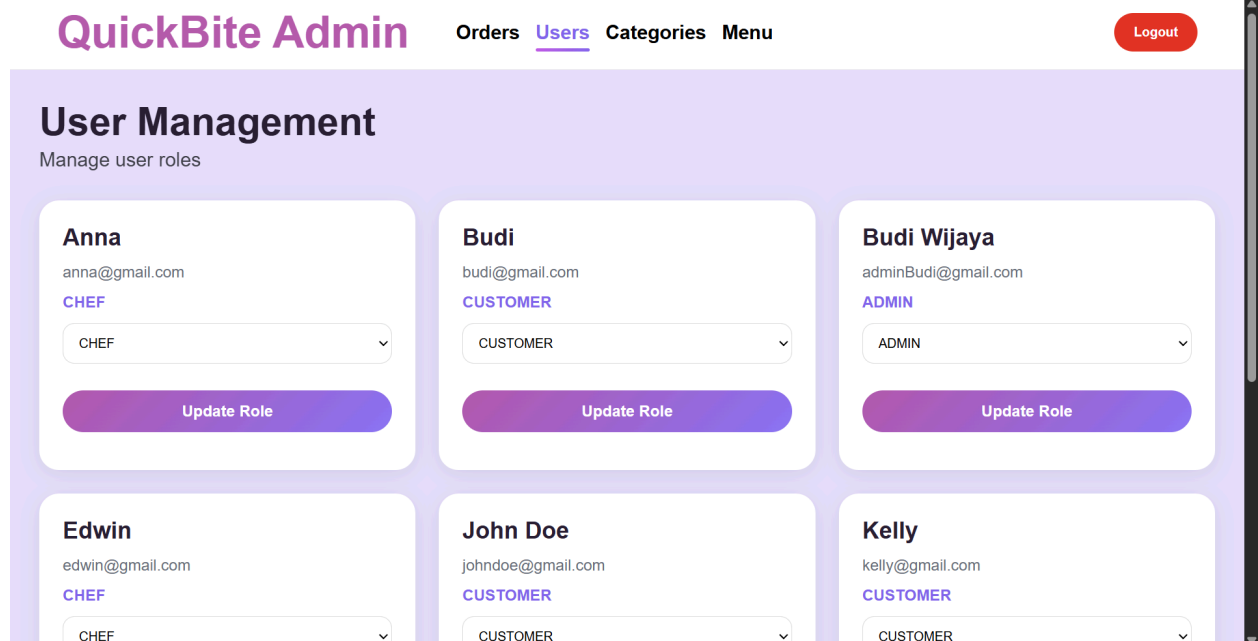
Pada halaman **Orders** di **Admin Page**, admin dapat melihat seluruh daftar pesanan yang tersedia dalam sistem. Setiap pesanan ditampilkan beserta detail statusnya. Untuk pesanan dengan status **Pending** dan belum memiliki chef yang ditugaskan, sistem menyediakan

dropdown yang berisi daftar chef yang tersedia. Admin dapat memilih salah satu chef dari dropdown tersebut dan menugaskannya ke pesanan yang dipilih dengan menekan tombol **Assign Chef**. Setelah proses ini dilakukan, pesanan akan otomatis terhubung dengan chef yang dipilih dan status penugasan akan diperbarui sesuai sistem.

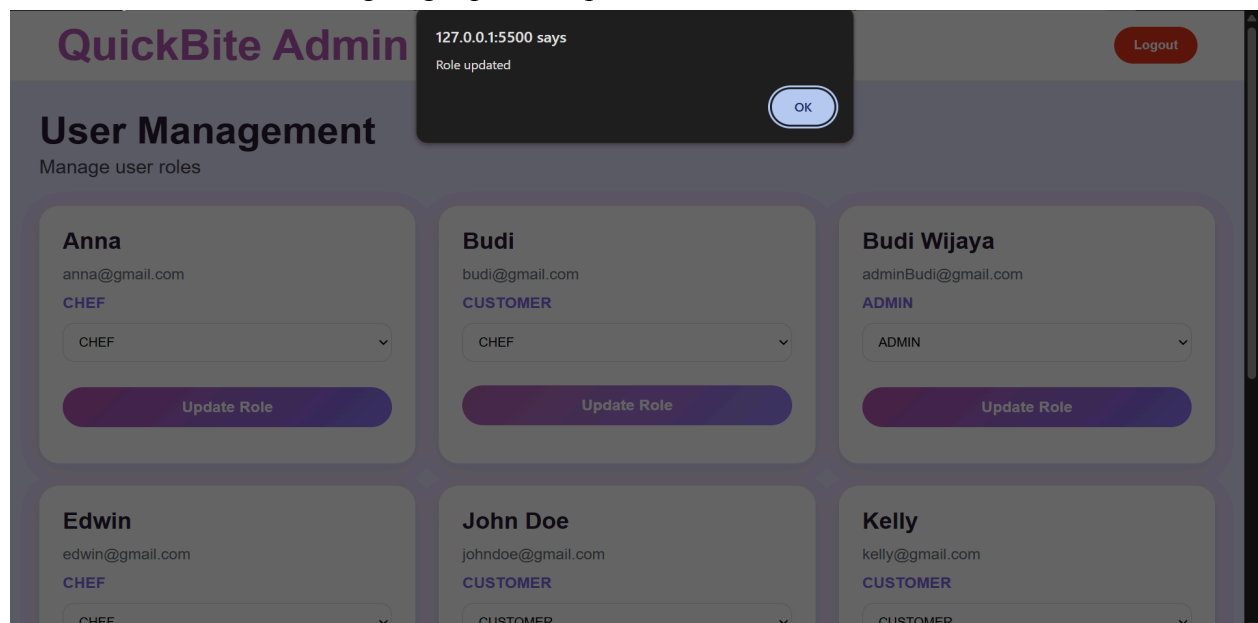


Pada halaman **Users** di **Admin Page**, admin dapat melihat seluruh daftar pengguna yang terdaftar dalam sistem. Setiap pengguna ditampilkan beserta informasi dasar seperti nama, email, dan role saat ini. Admin memiliki hak untuk mengubah role setiap pengguna sesuai kebutuhan dengan memilih role baru melalui dropdown yang tersedia. Setelah perubahan

dilakukan, admin dapat menekan tombol **Update Role** untuk memperbarui data pengguna. Sistem akan secara otomatis menyimpan perubahan dan memperbarui role pengguna di dalam database.



Perubahan role pengguna berhasil dilakukan, misalnya dari **Customer menjadi Chef** untuk user **Budi**, dan data role langsung diperbarui pada sistem.



Pada halaman **Categories** di **Admin Page**, admin dapat melihat seluruh daftar kategori yang tersedia dalam sistem. Setiap kategori ditampilkan beserta informasi dasar seperti nama dan deskripsi. Selain itu, admin juga dapat menambahkan kategori baru melalui fitur **Create**

Category dengan mengisi data yang diperlukan, kemudian menyimpannya ke dalam sistem. Setelah berhasil ditambahkan, kategori baru akan langsung ditampilkan pada daftar kategori.

The screenshot shows the 'QuickBite Admin' interface with the 'Categories' menu item selected. The main heading is 'Category Management' with a subtitle 'Create new categories'. Below this is a form with two input fields: 'Category Name' and 'Category Description'. A purple 'Create Category' button is positioned below the description field. At the bottom, there are three category cards: 'Desserts' (Delightful sweet treats and pastries perfect for concluding your meal or enjoying as a standalone indulgence.), 'Drinks' (A refreshing selection of cold and hot beverages, ranging from traditional teas and coffees to tropical fruit juices.), and 'Indonesian Food' (Authentic Indonesian dishes rich in traditional herbs and spices, delivering a unique and vibrant blend of savory, sweet, and spicy flavors from across the archipelago).

Kategori yang berhasil dibuat akan langsung tersimpan dalam sistem dan muncul pada daftar kategori tanpa perlu refresh.

This screenshot shows the same 'Category Management' page as before, but with a success message overlay. The message box, located at the top center, reads '127.0.0.1:5500 says Category created successfully' and has an 'OK' button. The form fields now contain 'Korean Food' and 'A variety of traditional and modern Korean dishes featuring bold flavors, balanced seasoning, and signature ingredients such as kimchi, rice, noodles, and grilled specialties.' The 'Create Category' button is still visible below the description field. The category cards at the bottom remain the same.

Pada halaman **Menu** di **Admin Page**, admin dapat melihat seluruh daftar menu yang tersedia dalam sistem. Admin juga dapat menambahkan menu baru melalui fitur **Create Menu**. Selain itu, admin dapat melakukan **Edit Menu** untuk memperbarui informasi menu yang

sudah ada. Setiap menu juga memiliki status ketersediaan yang dapat diubah melalui fitur **Enable** atau **Disable**, sehingga menu dapat diaktifkan atau dinonaktifkan sesuai kebutuhan operasional.

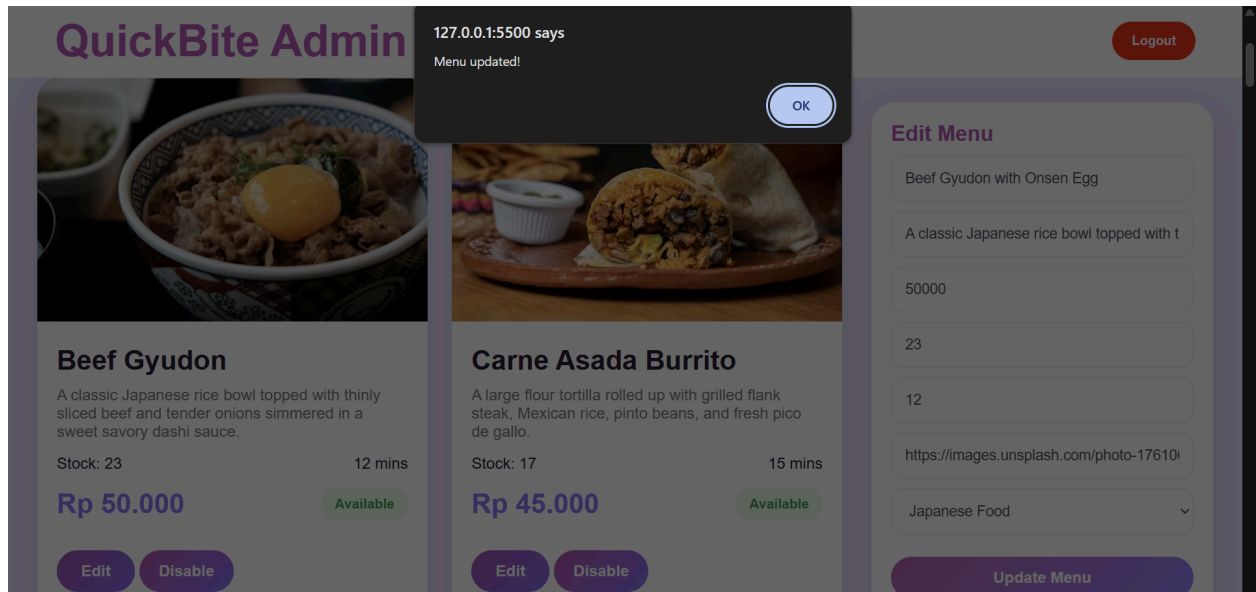
The screenshot shows the 'QuickBite Admin' dashboard with the 'Menu' tab selected. The main section is 'Menu Management' with the subtitle 'Manage menu availability'. Below this is the 'All Menu' section, which includes a prompt to 'Browse all available food and drinks' and a row of category buttons: 'All', 'Desserts', 'Drinks', 'Indonesian Food', 'Italian Food', 'Japanese Food', and 'Korean Food'. Below the buttons are two food images: a bowl of ramen with a soft-boiled egg and a burrito. On the right side, there is a 'Create New Menu' form with the following fields: 'Menu Name', 'Description', 'Price', 'Stock', 'Cooking Time (mins)', 'Image URL', and a 'Select Category' dropdown menu. A 'Create Menu' button is at the bottom of the form. A 'Logout' button is located in the top right corner.

Menu baru berhasil dibuat melalui fitur **Create Menu** dengan mengisi informasi yang diperlukan seperti nama, deskripsi, harga, stok, waktu memasak, gambar, dan kategori, kemudian menyimpannya ke dalam sistem. Data menu tersebut langsung ditambahkan ke daftar menu yang tersedia.

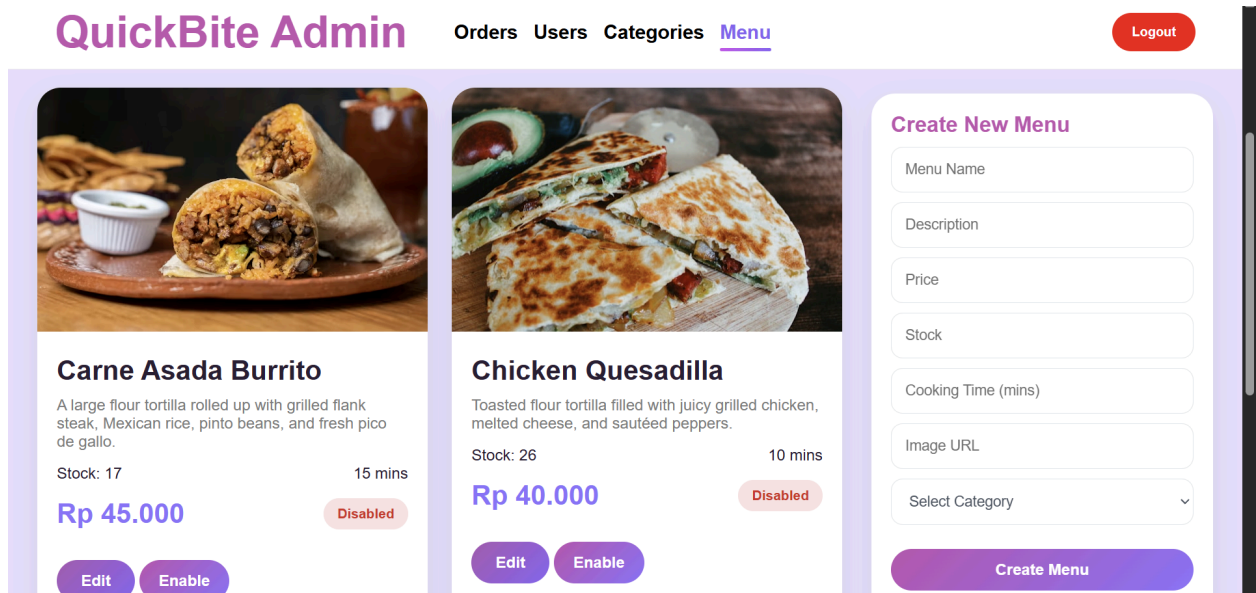
This screenshot shows the same 'QuickBite Admin' dashboard as the previous one, but with a dark notification banner at the top center that reads '127.0.0.1:5500 says Menu created successfully' with an 'OK' button. The 'Create New Menu' form on the right is now filled with the following data: 'Menu Name' is 'Cheesy Buldak Chicken Ramyeon', 'Description' is 'Springy Korean instant noodles tossed in a', 'Price' is '38000', 'Stock' is '24', 'Cooking Time (mins)' is '10', 'Image URL' is 'https://media.istockphoto.com/id/22022937', and 'Select Category' is 'Korean Food'. The 'Create Menu' button is still visible at the bottom of the form. The 'All Menu' section and category buttons remain the same. The 'Logout' button is still in the top right corner.

Admin dapat memperbarui informasi menu yang sudah ada. Ketika tombol **Edit Menu** ditekan, data menu akan otomatis ditampilkan pada form sehingga admin dapat langsung mengubah detail seperti nama menu, deskripsi, harga, stok, estimasi waktu memasak,

gambar, dan kategori menu. Setelah perubahan selesai dilakukan, admin dapat menekan tombol **Update Menu** untuk menyimpan perubahan ke dalam sistem.



Ketika tombol **Disable** ditekan, status menu akan berubah menjadi **Disabled**. Menu dengan status disabled akan disembunyikan dari tampilan semua role pengguna dan tidak lagi tersedia untuk dipilih atau dilihat. Selain itu, tombol akan berubah menjadi **Enable**, yang dapat digunakan untuk mengaktifkan kembali menu. Saat menu di-enable, statusnya kembali menjadi **Available** dan akan muncul kembali di sistem. Fitur ini digunakan untuk mengatur ketersediaan menu secara dinamis tanpa menghapus data dari sistem.



Tampilan menu pada role **Guest** menunjukkan perubahan setelah dilakukan penghapusan beberapa item. Sebelumnya, menu **Carne Asada Burrito** dan **Chicken Quesadilla** masih muncul dalam daftar menu yang tersedia.

Mexican Food

Vibrant and festive dishes packed with bold flavors, featuring a perfect combination of zesty lime, fresh cilantro, savory meats, and warm corn tortillas.

[All](#)[Desserts](#)[Drinks](#)[Indonesian Food](#)[Italian Food](#)[Japanese Food](#)[Korean Food](#)[Mexican Food](#)

Carne Asada Burrito

A large flour tortilla rolled up with grilled flank steak, Mexican rice, pinto beans, and fresh pico de gallo.

Mexican Food

15 mins

Stock: 17

Rp 45.000



Chicken Quesadilla

Toasted flour tortilla filled with juicy grilled chicken, melted cheese, and sautéed peppers.

Mexican Food

10 mins

Stock: 26

Rp 40.000



Chicken Tacos

Three crispy taco shells filled with grilled chicken, fresh lettuce, and zesty salsa sauce.

Mexican Food

10 mins

Stock: 34

Rp 38.000

Namun setelah dilakukan penghapusan, kedua menu tersebut tidak lagi ditampilkan pada halaman menu, sehingga daftar menu yang terlihat hanya berisi item yang masih aktif di dalam sistem.

Mexican Food

Vibrant and festive dishes packed with bold flavors, featuring a perfect combination of zesty lime, fresh cilantro, savory meats, and warm corn tortillas.

[All](#)[Desserts](#)[Drinks](#)[Indonesian Food](#)[Italian Food](#)[Japanese Food](#)[Korean Food](#)[Mexican Food](#)

Chicken Tacos

Three crispy taco shells filled with grilled chicken, fresh lettuce, and zesty salsa sauce.

Mexican Food

10 mins

Stock: 34

Rp 38.000